Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

Prerequisites:

-> Google Colab Account

Setup Instructions -> Open Google Colab -> Install Required Libraries

```
Project Structure:

sentiment-analysis-pipeline/
├── app.py                # Flask application setup and routing
├── pipeline_model.py     # Sentiment analysis pipeline logic
└── templates/
    ├── index.html        # Homepage with text input
    └── result.html
```

Project Structure:

sentiment-analysis-pipeline/ ├── app.py # Flask application setup and routing ├── pipeline_model.py # Sentiment analysis pipeline logic └── templates/ ├── index.html # Homepage with text input └── result.html

```
# # sentiment_model.py
# from transformers import pipeline

# # Load pre-trained sentiment-analysis pipeline
# nlp = pipeline('sentiment-analysis')

# def predict_sentiment(text):
#     result = nlp(text)[0]
#     return result



# sentiment_model.py
from transformers import pipeline

# Load pre-trained pipelines
sentiment_nlp = pipeline('sentiment-analysis')
emotion_nlp = pipeline('text-classification', model="j-hartmann/emotion-english-distilroberta-base")

def predict_sentiment(text):
    result = sentiment_nlp(text)[0]
    return result

def predict_emotion(text):
    result = emotion_nlp(text)[0]
    return result
```

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f (https://huggin
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%                                                 629/629 [00:00<00:00, 30.0kB/s]
model.safetensors: 100%                                           268M/268M [00:01<00:00, 202MB/s]
tokenizer_config.json: 100%                                       48.0/48.0 [00:00<00:00, 1.71kB/s]
vocab.txt: 100%                                                   232k/232k [00:00<00:00, 5.61MB/s]
config.json: 100%                                                 1.00k/1.00k [00:00<00:00, 41.2kB/s]
pytorch_model.bin: 100%                                           329M/329M [00:02<00:00, 184MB/s]
tokenizer_config.json: 100%                                       294/294 [00:00<00:00, 11.8kB/s]
vocab.json: 100%                                                  798k/798k [00:00<00:00, 21.4MB/s]
merges.txt: 100%                                                  456k/456k [00:00<00:00, 6.77MB/s]
tokenizer.json: 100%                                              1.36M/1.36M [00:00<00:00, 32.2MB/s]
special_tokens_map.json: 100%                                     239/239 [00:00<00:00, 15.5kB/s]
```

```
# sentiment_model.py
from transformers import pipeline
```

pipeline: Importing the pipeline function from the transformers library, which allows you to easily load pre-trained models for various NLP tasks.

```
# Load pre-trained pipelines
sentiment_nlp = pipeline('sentiment-analysis')
emotion_nlp = pipeline('text-classification', model="j-hartmann/emotion-english-distilroberta-base")
```

Load Pre-trained Pipelines:

-> sentiment_nlp: Initializes a pre-trained sentiment analysis pipeline. This pipeline can be used to determine the sentiment (e.g., positive or negative) of a given text.

-> emotion_nlp: Initializes a pre-trained text classification pipeline using the j-hartmann/emotion-english-distilroberta-base model, which can be used to classify the emotion conveyed in a given text (e.g., happiness, sadness).

```
def predict_sentiment(text):
    result = sentiment_nlp(text)[0]
    return result
```

Define Sentiment Prediction Function:

-> predict_sentiment(text): Defines a function that takes a text input and predicts its sentiment.

-> result = sentiment_nlp(text)[0]: Uses the sentiment_nlp pipeline to analyze the sentiment of the provided text. The result is a list of predictions, and [0] retrieves the first (and only) prediction.

-> return result: Returns the sentiment analysis result.

```
def predict_emotion(text):
    result = emotion_nlp(text)[0]
    return result
```

Define Emotion Prediction Function:

-> predict_emotion(text): Defines a function that takes a text input and predicts its emotion.

-> result = emotion_nlp(text)[0]: Uses the emotion_nlp pipeline to classify the emotion conveyed in the provided text. The result is a list of predictions, and [0] retrieves the first (and only) prediction.

-> return result: Returns the emotion classification result.

It will look as below

```
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

config.json: 100% 629/629 [00:00<00:00, 30.0kB/s]

model.safetensors: 100% 268M/268M [00:01<00:00, 202MB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.71kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 5.61MB/s]

config.json: 100% 1.00k/1.00k [00:00<00:00, 41.2kB/s]

pytorch_model.bin: 100% 329M/329M [00:02<00:00, 184MB/s]

tokenizer_config.json: 100% 294/294 [00:00<00:00, 11.8kB/s]

vocab.json: 100% 798k/798k [00:00<00:00, 21.4MB/s]

merges.txt: 100% 456k/456k [00:00<00:00, 6.77MB/s]

tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 32.2MB/s]

special_tokens_map.json: 100% 239/239 [00:00<00:00, 15.5kB/s]

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.